SPARFUCHS

Corporation®

Warp-speed delivery. Fort Knox security. 30 days.

ADMIN GUIDE | VOLUME II

Sparfuchs QA Administrator Guide

For administrators provisioning, configuring, securing, and operating Sparfuchs QA

Version	1.0
Release date	April 22, 2026
Audience	For administrators provisioning, configuring, securing, and operating Sparfuchs QA
Authors	Sparfuchs Corporation Platform Team
Classification	Corporate Confidential

No part of this document, or any associated materials, may be reproduced, distributed, transmitted, displayed, published, or broadcast in any form or by any means, including electronic, mechanical, photocopying, recording, or otherwise, without the prior written permission of Sparfuchs Corporation.

Unauthorized use of this material may violate copyright laws, trademark laws, and other applicable regulations. All trademarks and registered trademarks mentioned herein are the property of their respective owners.

How to use this guide

This Administrator Guide is written for the engineer or operator who owns Sparfuchs QA for a team or organization. It is not a substitute for the companion User Guide, it extends it with everything an administrator needs to provision the toolkit safely, tune it for a specific organization, and keep it running in a compliant posture.

If you are going to be the person answering the question "how do we set Sparfuchs QA up here?", read chapters 1 through 5 front to back. If you are responding to an incident or a compliance request, skip to chapters 7 and 8.

Conventions

Administrator. The owner of Sparfuchs QA for a team. Has write access to *config/models.yaml*, the OS keychain, and the *.claude/agents/* directory.

Operator. The end-user who runs reviews. See the User Guide for their view.

Target repo. An external repository being reviewed. Read-only from the orchestrator's point of view.

Cautions appear in red boxes. Tips appear in green. Informational notes in blue. All commands are copy-safe plain ASCII.

Related documents

- **User Guide**, the companion volume with operator workflows.
- *docs/QA-ARCHITECTURE.md* in the sparfuchs-qa repo, for deeper source references.
- *docs/CAVEAT-EMPTOR.md*, the honest gaps assessment.
- *config/models.yaml*, the live configuration, which is the ground truth.

Table of contents

How to use this guide 2
List of figures 6
1. Introduction and architecture 7
1.1 Deployment model in one diagram . . . 7
1.2 System architecture . . . 7
1.3 Data flow . . . 9
1.4 Roles and permissions . . . 9
2. System requirements 10
2.1 Supported platforms . . . 10
2.2 Software prerequisites . . . 10
2.3 Capacity planning . . . 10
2.4 Network requirements . . . 10
3. Installation and deployment 12
3.1 Fresh installation . . . 12
3.2 Automated versus manual installs . . . 12
3.3 Pre- and post-install checklist . . . 12
3.4 Rollback and uninstall . . . 13
3.5 Agent framework setup . . . 13
3.6 Tool registry . . . 13
3.7 Memory and persistence backends . . . 14
4. Initial configuration 15
4.1 models.yaml in detail . . . 15
4.2 Model tiers . . . 15
4.3 Per-agent configuration overrides . . . 15
4.4 Token budget presets . . . 16
4.5 Safety guardrails . . . 16
4.6 External integrations . . . 16
5. Credential and secrets management 18
5.1 Resolution order and lifecycle . . . 18
5.2 LLM provider keys, by platform . . . 18
5.3 Verifying keychain state . . . 19
5.4 Gemini CLI OAuth alternative . . . 19

5.5 Purging credentials . . . 19
5.6 Test credential profiles . . . 19
5.7 Auth strategies . . . 20
5.8 Secrets redaction . . . 20
6. User, role, and access management 21
6.1 Recommended role matrix . . . 21
6.2 Day-one hardening checklist . . . 21
6.3 SSO and identity . . . 22
6.4 Multi-factor authentication . . . 22
6.5 API key management lifecycle . . . 22
6.6 Agent sandboxing and containment . . . 23
6.7 What agents should never be allowed to do . . . 24
7. Monitoring, logging, and observability 25
7.1 What gets measured . . . 25
7.2 Dashboards and external tools . . . 25
7.3 Alerting thresholds . . . 26
7.4 Agent-specific monitoring . . . 26
7.5 Anomaly detection . . . 26
8. Security, compliance, and governance 27
8.1 Security model summary . . . 27
8.2 Encryption and data handling . . . 27
8.3 Audit logging . . . 27
8.4 Compliance posture . . . 27
8.5 Known gaps and what to do about them today . . . 28
8.6 Risk assessment and escalation . . . 28
8.7 Oversight policies for agentic behavior . . . 29
8.8 Anti-hallucination layers . . . 29
9. Maintenance and operations 31
9.1 Maintenance cadence . . . 31
9.2 Backup and restore . . . 31
9.3 Updates and patching . . . 31
9.4 Zero-downtime rollouts . . . 32
9.5 Scaling considerations . . . 32
9.6 Disaster recovery . . . 32
9.7 Updating agent logic and prompts . . . 32

10. API, integration, and extensibility 33
10.1 Artifact formats . . . 33
10.2 The JSON handoff contract . . . 33
10.3 Writing a new agent . . . 33
10.4 Writing a new adapter . . . 34
10.5 Event streams for dashboards . . . 34
10.6 Webhooks and integrations . . . 34
10.7 Plugins and custom tools . . . 34
11. Advanced troubleshooting and diagnostics 35
11.1 Log analysis playbook . . . 35
11.2 Common failure scenarios . . . 35
11.3 Performance bottlenecks . . . 35
11.4 Debug modes . . . 36
11.5 Support bundle creation . . . 36
12. Appendices 37
Appendix A. Sample models.yaml . . . 37
Appendix B. Reference tables . . . 37
Appendix C. Audit evidence checklist . . . 38
Appendix D. License information . . . 38
Appendix E. Vendor contacts . . . 38
Appendix F. Changelog . . . 38
Appendix G. Hardening roadmap . . . 39
Appendix H. Documentation maintenance plan . . . 39

List of figures

#	Figure	Chapter
1	Figure 1.1 Deployment topology	Chapter 1
2	Figure 1.2 Sparfuchs QA, system overview	Chapter 1
3	Figure 1.3 Recommended operator permissions	Chapter 1
4	Figure 5.1 Credential resolution and lifecycle	Chapter 5
5	Figure 6.1 Recommended operator permissions	Chapter 6
6	Figure 7.1 Observability pipeline	Chapter 7
7	Figure 8.1 Governance and escalation	Chapter 8
8	Figure 8.2 Anti-hallucination layers	Chapter 8

1. Introduction and architecture

Sparfuchs QA is a TypeScript multi-agent quality toolkit architected as a workstation-local tool. There is no phone-home, no server-side data retention, and no shared database. Source code and findings never leave your perimeter unless you explicitly enable an external LLM provider. For air-gapped and regulated deployments, set *dataClassification*: *restricted* to disable all external API providers in one line of config (chapter 4.4). This chapter gives the architectural grounding an administrator needs to reason about deployment, security, and governance.

1.1 Deployment model in one diagram

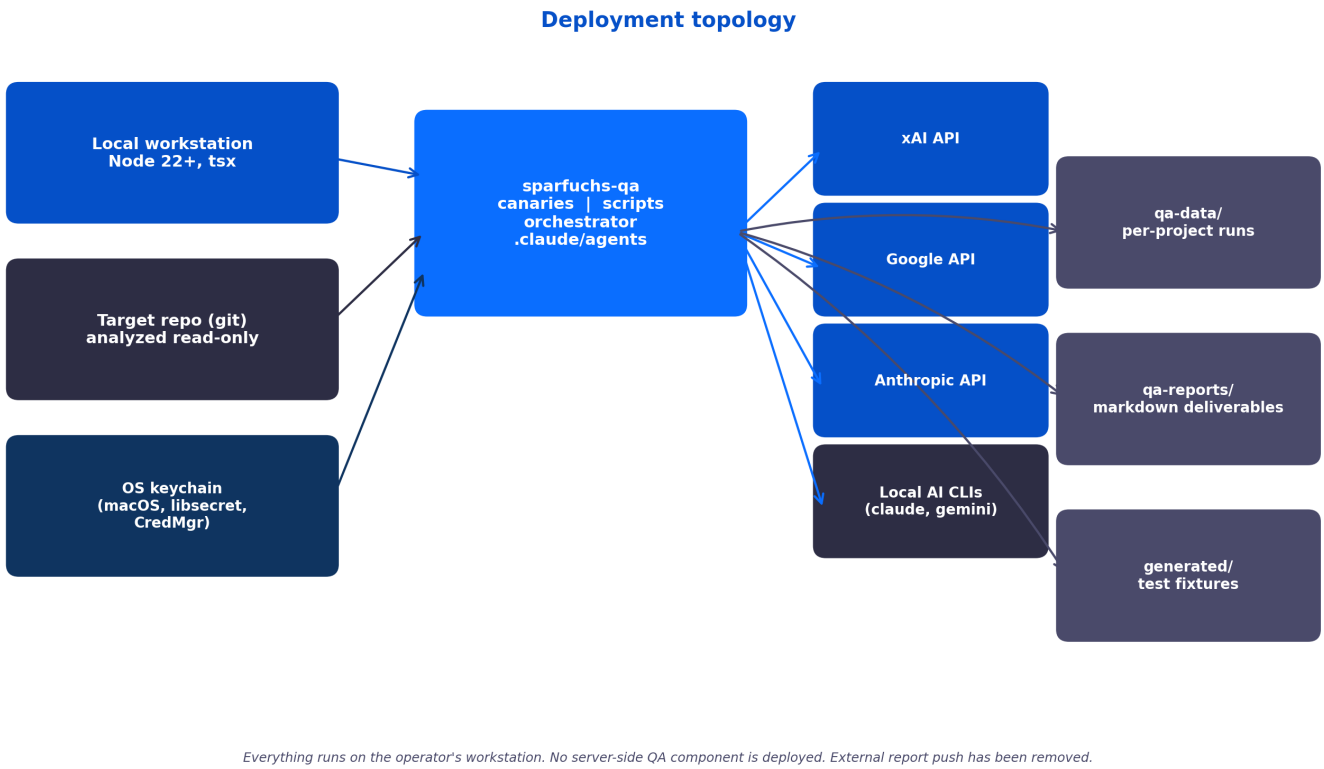

Figure 1.1 Deployment topology.

Because there is no persistent service and no shared database, the attack surface available to an external party is minimal. The workstation is the boundary, and hardening Sparfuchs QA is largely a matter of hardening the operator's local environment: keychain hygiene, filesystem permissions, credential lifecycle, and network egress controls. Everything an administrator needs to enforce this boundary lives in the chapters that follow.

Tip. For the compliance questionnaire version of this claim: no inbound ports are opened on the operator's workstation, no outbound traffic flows except to the LLM providers the operator has explicitly enabled, and no data is persisted outside the operator's chosen working directory.

1.2 System architecture

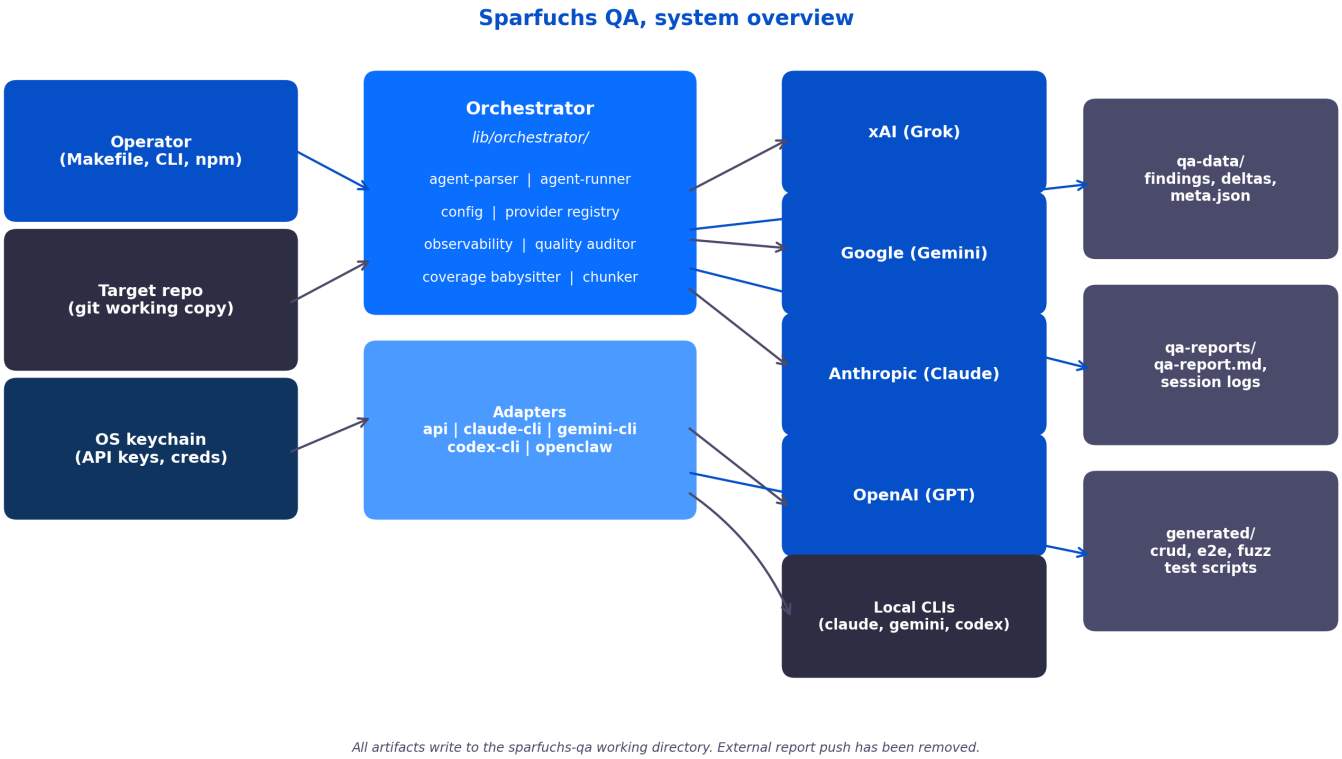

Figure 1.2 Sparfuchs QA, system overview.

Component	Source path	Role
Canary runner	canaries/index.ts	Auto-discovers **.canary.ts* files, prints a JSON summary.
Orchestrator	lib/orchestrator/	Parses agents, runs them across providers, audits outputs.
Agent parser	lib/orchestrator/agent-parser.ts	Validates frontmatter, computes SHA-256 hashes.
Agent runner	lib/orchestrator/agent-runner.ts	Executes a single agent with retries and fallbacks.
Config loader	lib/orchestrator/config.ts	Loads *config/models.yaml*, resolves keys, enforces classification.
Credential store	lib/orchestrator/credential-store.ts	OS keychain integration for encrypted API key storage.
Credential manager	lib/credentials/credential-manager.ts	Test credential profiles, loaded via *SPARFUCHS_CRED_FILE*.
Observability	lib/orchestrator/observability.ts	Duration, tokens, tool calls, errors per agent.
Quality auditor	lib/orchestrator/quality-auditor.ts	Post-run validation, hallucination-risk scoring.
Findings manager	scripts/qa-findings-manager.ts	JSONL registry with lifecycle states.
Flaky tracker	scripts/flaky-test-tracker.ts	Candidate at 2 flips, confirmed at 5.
Delta reporter	scripts/qa-delta-report.ts	New / recurring / remediated across runs.

Component	Source path	Role
Evolution engine	scripts/qa-evolve-v2.ts	Suggests threshold evolution from history.
File audit cache	scripts/file-audit-cache.ts	Incremental auditing per file per commit.
Package verifier	scripts/package-verify.ts	SBOM + provenance attestation checks.

1.3 Data flow

The orchestrator's read path and write path are strictly separated. Reads come from the target repo (source files, *git log*, *git diff*) and from the sparfuchs-qa workspace (*config/models.yaml*, *.claude/agents/*, previous run artifacts). Writes go only to the sparfuchs-qa workspace under *qa-data/*, *qa-reports/*, and *generated/*. The target repo is never modified, except for the brief window when specialist agent markdowns are copied into *.claude/agents/* and then removed at cleanup.

1.4 Roles and permissions

Sparfuchs QA has no built-in RBAC layer. Permissions are enforced by the host OS: keychain access controls, filesystem permissions on *config/models.yaml* and the *.claude/agents/* directory, and the Claude Code *permission-mode* setting. The matrix below is the recommended operating model for a team.

Operator permissions for Sparfuchs QA

Action	Operator	Reviewer	Admin
Run make qa-quick	YES	YES	YES
Run make qa-review	YES	YES	YES
Store API keys in keychain	YES	NO	YES
Modify .claude/agents/	NO	NO	YES
Edit config/models.yaml	NO	NO	YES
Regenerate agent-hashes.json	NO	NO	YES
Delete qa-data runs	YES	YES	YES
Set dataClassification=restricted	NO	NO	YES

The tool has no built-in RBAC layer. These are recommended operating roles for a team using Sparfuchs QA.

Figure 1.3 Recommended operator permissions.

2. System requirements

Sparfuchs QA runs on the operator workstation and has modest hardware needs. The dominant requirement is network access to the chosen LLM provider(s) and a working OS keychain.

2.1 Supported platforms

Platform	Minimum	Notes
macOS	13 Ventura	*security* framework keychain.
Linux	Ubuntu 22.04, Debian 12, RHEL 9	Requires *libsecret* and *secret-tool*.
Windows	Windows 11, PowerShell 7	CredentialManager PowerShell module.

2.2 Software prerequisites

Requirement	Version	Check
Node.js	22 or newer	`node --version`
npm	10 or newer	`npm --version`
Git	2.30 or newer	`git --version`
Make	3.81 or newer	`make --version`
Playwright (optional)	Latest	`npx playwright --version`
Claude Code CLI (optional)	Any	`claude --version`
Gemini CLI (optional)	Any	`gemini --version`
Codex CLI (optional)	Any	`codex --version`

2.3 Capacity planning

The orchestrator is memory-bound, not CPU-bound. Typical run memory usage is a few hundred megabytes per agent. LLM latency dominates wall-clock time. For a dedicated shared-review workstation used by several operators, we recommend:

- 8 GB RAM minimum, 16 GB for parallel canary watch + review.
- 10 GB free disk per project, for *qa-data/* history.
- 100 Mbps or better network egress to the LLM providers.
- Unrestricted access to the OS keychain binary (*security*, *secret-tool*, *Get-StoredCredential*).

2.4 Network requirements

The orchestrator does not host any network services. It only initiates outbound connections:

Destination	Protocol	Reason
api.anthropic.com	HTTPS	Claude API (anthropic provider)
api.openai.com	HTTPS	GPT API (openai provider)
api.x.ai	HTTPS	Grok API (xai provider)
generativelanguage.googleapis.com	HTTPS	Gemini API (google provider)
github.com	HTTPS	Only for the git clone during initial install; optional after.
registry.npmjs.org	HTTPS	Only during *npm install*.

Note. For air-gapped deployments, set *dataClassification*: *restricted* to disable API providers entirely. The CLI providers (claude, gemini, codex) can work offline if the operator's CLI auth is cached and the model runs locally (e.g., *openclaw*). Air-gapped operation is a supported but uncommon deployment.

3. Installation and deployment

There is only one deployment model: clone the repo onto the operator's workstation, install dependencies, and store credentials. This chapter documents the steps, the rollback procedure, and the two out-of-the-box helper scripts.

3.1 Fresh installation

3.1.1 Prerequisites

- Node 22 or newer on PATH.
- Git with access to the sparfuchs-qa repository.
- Admin rights on the workstation if the OS keychain requires them for initial setup.

3.1.2 Procedure

1. Clone: *git clone https://github.com/Sparfuchs-Pro/sparfuchs-qa.git*.
2. Change into the directory: *cd sparfuchs-qa*.
3. Run the bootstrap script: *bash setup-qa-complete.sh*. This validates repo state, fixes hook permissions, and runs *npm ci*.
4. Alternative: *make qa-setup* to just run *npm install*.
5. Store at least one API credential in the OS keychain (chapter 5).
6. Run *make qa-quick* to verify installation. 15 of 15 canaries should pass.

```
git clone https://github.com/Sparfuchs-Pro/sparfuchs-qa.git
cd sparfuchs-qa
bash setup-qa-complete.sh
make qa-quick
```

3.2 Automated versus manual installs

setup-qa-complete.sh is the automated path. It is idempotent, repo-relative, and safe to re-run. It does not rewrite tracked files, it does not clone unrelated repositories, and it does not auto-push git changes. If you want a fully manual install for audit reasons, skip the script and run *npm ci* directly.

Warning. The old destructive installer has been replaced. If you have a snapshot of the pre-2026 repo with the previous installer, do not run it. Use *setup-qa-complete.sh* exclusively.

3.3 Pre- and post-install checklist

Pre-install	Verified
Node 22+ on PATH	
Git access to sparfuchs-qa repo	
Internet reachable for *npm install*	
Keychain tool installed (security / secret-tool / CredMgr)	

Pre-install	Verified
Post-install	Verified
make qa-quick prints 15/15 passed	
make qa-keys-check lists at least one key	
Trial review against a sample repo finishes	
Cleanup trap correctly removes deployed agents	

3.4 Rollback and uninstall

Uninstalling is non-destructive by design. Use the bundled uninstall script to remove local generated artifacts:

```
bash uninstall-qa-complete.sh
```

This removes *qa-data/*, *qa-reports/*, *generated/*, and the *node_modules/* directory. It does NOT remove source files, and it does NOT remove the OS keychain entries. To purge keychain entries, see section 5.5.

3.5 Agent framework setup

Specialist agents live as markdown files in *.claude/agents/*. The current repo ships 44 agent markdown files: 41 standard QA specialists, 2 optional documentation agents (*architecture-doc-builder*, *training-system-builder*), and 1 local workflow helper (*session-cleanup*). Count the *.md* files in *.claude/agents/* at any time for an authoritative number; the agent set grows as new specialists are added. The agent parser validates each file on startup, computes a SHA-256 hash, and compares it to *config/agent-hashes.json*.

1. After any intentional agent edit, run *make qa-hashes-update*.
2. Agents are auto-deployed to target repos at run start and removed on cleanup.
3. If you add a new agent markdown, include it in the standard deployment list in *scripts/qa-review-remote.sh*.

3.6 Tool registry

Each agent declares the tools it may use in its frontmatter. The orchestrator enforces this at runtime. The full tool set is: *Read*, *Write*, *Edit*, *Bash*, *Grep*, *Glob*, *WebFetch*, plus any custom MCP tools declared in *config/*.

Tool	Typical users	Admin control
Read	All agents	Cannot be disabled globally; file path jail applied.
Grep, Glob	All agents	Cannot be disabled; pattern limits via testability scanner.
Bash	Test runners, build verifier	Per-agent *disableBash*: *true* in overrides.
Write	None by default	Blocked in target repo; writes only to qa-reports/.

Tool	Typical users	Admin control
Edit	None	Blocked in target repo.
WebFetch	Reference doc verifier only	Egress rules apply; see section 7.3.

3.7 Memory and persistence backends

Sparfuchs QA does not require a database. All state is JSON / JSONL on disk:

Artifact	Location	Purpose
Run bundle	qa-data/<project>/runs/<runId>/	Per-run JSON artifacts (meta, findings, delta, coverage, quality audit).
Findings registry	qa-data/<project>/findings-registry.jsonl	Long-lived lifecycle state.
File audit cache	qa-data/<project>/file-audit.json	Incremental audit tracking.
Agent hashes	config/agent-hashes.json	Integrity validation baseline.
Flaky tracker	qa-data/<project>/flaky.json	Per-test flip counts.

4. Initial configuration

The two configuration touchpoints that matter most are *config/models.yaml* (providers, tiers, classification) and the OS keychain (credentials). Everything else has sensible defaults that only a power user should need to touch.

4.1 models.yaml in detail

```
defaultProvider: xai
coverageStrategy: balanced # sweep | balanced | thorough | exhaustive
dataClassification: public # public | internal | restricted
redactSecrets: true
# approvedProviders: [anthropic] # required when classification = internal

providers:
xai: { type: api, enabled: true, apiKeyEnvVar: XAI_API_KEY }
google: { type: api, enabled: true, apiKeyEnvVar: GOOGLE_GENERATIVE_AI_API_KEY }
anthropic: { type: api, enabled: true, apiKeyEnvVar: ANTHROPIC_API_KEY }
openai: { type: api, enabled: true, apiKeyEnvVar: OPENAI_API_KEY }
claude-cli: { type: cli, enabled: auto, binary: claude }
gemini-cli: { type: cli, enabled: auto, binary: gemini }
codex-cli: { type: cli, enabled: auto, binary: codex }
openclaw: { type: cli, enabled: auto, binary: openclaw }

fallbackChain: [xai, google, anthropic, claude-cli, gemini-cli, codex-cli]
```

4.2 Model tiers

Agent frontmatter declares a tier, and *models.yaml* maps each tier to a specific model per provider. Tiers are a cost / depth knob:

Agent frontmatter	Internal tier	Typical usage
opus	heavy	Deep analysis, security reviewer, release gate synthesizer.
sonnet	mid	Standard code reviewer, performance reviewer, compliance.
haiku	light	Fixture generation, doc review, simple classification.

4.3 Per-agent configuration overrides

Overrides are the right knob for three cases: lock an agent to a trusted provider, increase its step budget, or tighten its tool access.

```
agentOverrides:
security-reviewer:
provider: xai # honesty-critical, prefer Grok
ui-intent-verifier:
provider: xai
disableBash: true # read-only
observability-auditor:
provider: xai
maxSteps: 75
```

```
ref-doc-verifier:
provider: xai
maxSteps: 100
```

4.4 Token budget presets

The *tokenBudget.presets* block lets the administrator define named agent sets. Operators can select a preset with a future *--preset* flag; today the default is *full*. Pricing is informational only, it is not enforced.

Preset	Contents	When to use
full	All agents	Default. First-time onboarding, quarterly audits.
standard	~18 agents (build, security, rbac, compliance, etc.)	Pre-PR review on well-understood projects.
lite	8 agents (build, security, test-runner, release gate, ...)	Rapid iteration, tight token budget.

4.5 Safety guardrails

The *.claude/hooks/* directory ships six hardening hooks that fire inside the Claude Code session. An administrator should review them, and may add additional hooks for organization-specific policies.

Hook	Fires on	Effect
session-start.sh	Session start	Prints the session preamble; pins the agent policy.
block-dangerous-commands.sh	Pre-tool-use (Bash)	Denies destructive commands (*rm -rf /*, *git push --force*, etc.).
scan-secrets.sh	Pre-write	Scans outbound content for secret patterns.
protect-files.sh	Pre-write	Blocks writes to *.env*, **.pem*, **.key*, lockfiles, *.git/*.
warn-large-files.sh	Pre-read	Warns when a file exceeds the read window.
format-on-save.sh	Post-write	Runs a formatter after a permitted write.

4.6 External integrations

The toolkit ships with zero external integrations enabled by default. Optional integrations the administrator can add include:

- Log forwarding: pipe *RENDERER=json* events to a log aggregator.
- Secret managers: fetch keys into env vars before invocation.
- CI integration: run *make qa-quick* in CI; any failure exits non-zero.

- Dashboards: write a small consumer that reads *qa-data/* artifacts and renders them. The MFE dashboard referenced in *docs/QA-ARCHITECTURE.md* is a separate repo.

5. Credential and secrets management

Sparfuchs QA has two distinct credential tracks. Track one is LLM provider keys (xAI, Google, Anthropic, OpenAI, Gemini CLI OAuth). Track two is test credentials for authenticated target applications. This chapter covers both.

5.1 Resolution order and lifecycle

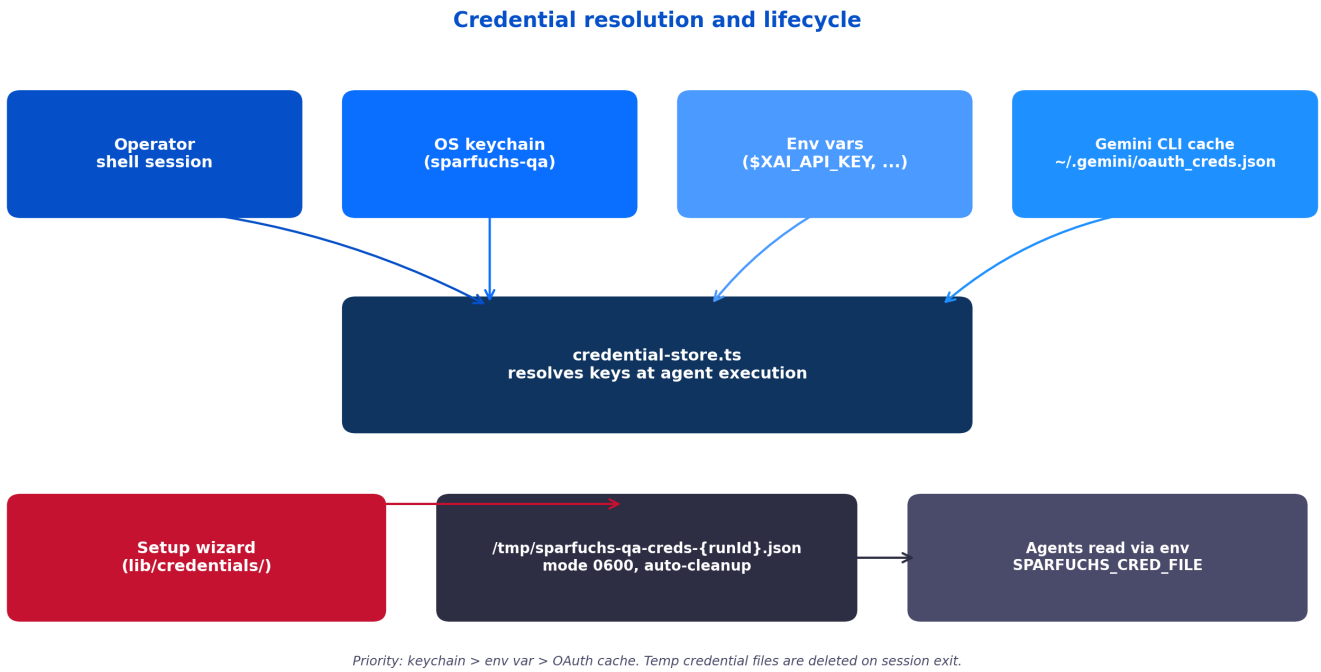

Figure 5.1 Credential resolution and lifecycle.

The resolution order for LLM keys is: OS keychain first, shell environment variable second, and the Gemini CLI OAuth cache (~/.gemini/oauth_creds.json) third, and only for the Gemini CLI adapter.

5.2 LLM provider keys, by platform

5.2.1 macOS, using the security framework

```
# Add / set
security add-generic-password -s sparfuchs-qa -a ANTHROPIC_API_KEY -w 'sk-ant-...'
security add-generic-password -s sparfuchs-qa -a OPENAI_API_KEY -w 'sk-...'
security add-generic-password -s sparfuchs-qa -a XAI_API_KEY -w 'xai-...'
security add-generic-password -s sparfuchs-qa -a GOOGLE_GENERATIVE_AI_API_KEY -w 'AIza...'
security add-generic-password -s sparfuchs-qa -a GEMINI_API_KEY -w 'AIza...'

# Update (delete + re-add)
security delete-generic-password -s sparfuchs-qa -a GEMINI_API_KEY
security add-generic-password -s sparfuchs-qa -a GEMINI_API_KEY -w 'new-value'

# Verify (prints to stdout)
security find-generic-password -s sparfuchs-qa -a GEMINI_API_KEY -w
```

5.2.2 Linux, using libsecret / secret-tool

```
echo 'sk-ant-...' | secret-tool store --label=sparfuchs-qa \
service sparfuchs-qa key ANTHROPIC_API_KEY
secret-tool lookup service sparfuchs-qa key ANTHROPIC_API_KEY
secret-tool clear service sparfuchs-qa key ANTHROPIC_API_KEY
```

5.2.3 Windows, using CredentialManager PowerShell

```
$s = ConvertTo-SecureString 'sk-ant-...' -AsPlainText -Force
New-StoredCredential -Target 'sparfuchs-qa-ANTHROPIC_API_KEY' \
-Password $s -Type Generic -Persist LocalMachine
```

5.3 Verifying keychain state

```
# Lists all <sparfuchs-qa> accounts present in the keychain
make qa-keys-check
```

Note. A fresh *make qa-keys-check* on an unprovisioned host prints *No keys in OS keychain*. That is expected. The error is only a problem if the operator has already stored a key and it does not appear here, which usually indicates a service-name mismatch.

5.4 Gemini CLI OAuth alternative

For operators who cannot store an API key (for example, regulated environments that forbid long-lived secrets), the Gemini CLI supports OAuth:

```
# One-time browser login. Saves to ~/.gemini/oauth_creds.json
gemini
```

Warning. OAuth cached creds have shorter lifetimes than API keys. If a long review aborts halfway through with an auth error, reauthenticate and resume. The orchestrator does not refresh OAuth automatically.

5.5 Purging credentials

To remove every sparfuchs-qa credential from the keychain:

```
# macOS
for a in ANTHROPIC_API_KEY OPENAI_API_KEY XAI_API_KEY \
GOOGLE_GENERATIVE_AI_API_KEY GEMINI_API_KEY; do
security delete-generic-password -s sparfuchs-qa -a $a 2>/dev/null
done

# Linux
for a in ANTHROPIC_API_KEY OPENAI_API_KEY XAI_API_KEY \
GOOGLE_GENERATIVE_AI_API_KEY GEMINI_API_KEY; do
secret-tool clear service sparfuchs-qa key $a 2>/dev/null
done
```

5.6 Test credential profiles

Test credentials are collected by the setup wizard and stored in the OS keychain under named profiles. The orchestrator looks them up by *NAME*:

```
# List named profiles
make qa-creds-list

# Store a new profile (interactive)
make qa-creds-store NAME=staging-admin
```

```
# Show a profile (redacted)
make qa-creds-show NAME=staging-admin

# Delete
make qa-creds-delete NAME=staging-admin
```

At run time, the wizard materializes the chosen profile to */tmp/sparfuchs-qa-creds-{runId}.json*, mode 0600. The orchestrator sets *SPARFUCHS_CRED_FILE* so agents can read it. The file is deleted on cleanup.

5.7 Auth strategies

Strategy	For	Wizard asks for
Email + Password	Firebase, standard login	Email, password, optional Firebase API key
API Token	Bearer or API keys	Token string
OAuth Token	Pre-obtained OAuth	Access token, refresh token, expiry
Basic Auth	HTTP Basic	Username, password
None	Public APIs	Nothing, no-op file

5.8 Secrets redaction

The *redactSecrets* setting in *models.yaml* is on by default. When on, the orchestrator scrubs ten regex patterns from every tool output and session log. These cover API keys, bearer tokens, private keys, and common connection strings.

Warning. Secrets redaction does **not** cover personally-identifiable information. This is a known gap (CAVEAT-EMPTOR section 4.2). If the target repo contains hardcoded PII (emails, phone numbers, SSNs), do not run reviews against it until PII redaction is added. The Admin Guide roadmap (section 11.2) tracks this.

6. User, role, and access management

Sparfuchs QA has no internal user database. Access control is delegated to the host operating system. This chapter documents the recommended RBAC model, how to apply it in practice, and how to delegate agentic actions safely.

6.1 Recommended role matrix

Operator permissions for Sparfuchs QA

Action	Operator	Reviewer	Admin
Run make qa-quick	YES	YES	YES
Run make qa-review	YES	YES	YES
Store API keys in keychain	YES	NO	YES
Modify .claude/agents/	NO	NO	YES
Edit config/models.yaml	NO	NO	YES
Regenerate agent-hashes.json	NO	NO	YES
Delete qa-data runs	YES	YES	YES
Set dataClassification=restricted	NO	NO	YES

The tool has no built-in RBAC layer. These are recommended operating roles for a team using Sparfuchs QA.

Figure 6.1 Recommended operator permissions.

Three roles cover 95% of teams: **Operator** (runs reviews), **Reviewer** (reads reports, can archive runs), and **Administrator** (owns the config and agent definitions). The matrix above is the default; adjust freely.

6.2 Day-one hardening checklist

Imagine you are standing up Sparfuchs QA for a team of five engineers across three shared workstations, and you want a posture you can defend in a security review without over-engineering. This checklist is the right first day.

6.2.1 Machine and filesystem

1. Ensure each workstation's disk uses full-volume encryption (FileVault, BitLocker, or LUKS). This compensates for the CRITICAL gap in chapter 8.5.1 until the planned v1.1 fix ships.

2. Set *config/models.yaml*, *config/agent-hashes.json*, and *.claude/agents/* to mode 0644 and own them by an admin group that operators cannot write to.

3. Create a per-operator *qa-data/* directory. Do not share *qa-data/* across users on a multi-tenant host.

6.2.2 Credentials and classification

1. Store provider API keys in the OS keychain under service *sparfuchs-qa*. Do not use shell environment variables for long-lived keys.

2. Keep a dedicated **read-only** provider account per operator. Use a separate **admin** provider account for automation, never installed on an operator workstation.

3. Set *dataClassification*: *restricted* on hosts that analyze regulated repositories. The only way to be certain source code does not reach an external API is to disable external APIs.

4. Never store production DB or admin credentials in the sparfuchs-qa keychain.

6.2.3 Auditability

1. Back up *findings-registry.jsonl* to a network share weekly. This is the only long-lived artifact that matters for a compliance audit.

2. Enforce *RENDERER=json* on shared workstations and pipe events into your log aggregator. That gives you a searchable record of who ran what review when.

3. Commit the operating policy (which roles exist, who holds admin keys, rotation cadence) to a private repo. Review it every quarter.

6.2.4 Runtime posture

1. Disable Bash via *agentOverrides* for the agents that have no reason to shell out. See chapter 6.7 for the allow-listed set we recommend.

2. Keep *redactSecrets*: *true* on. It is on by default; verify it stays on.

3. After any intentional agent edit, run *make qa-hashes-update* and commit the hashes file alongside the agent. Unexpected integrity warnings should be treated as a supply-chain incident.

Tip. If you only have time for four things on day one, do this: encrypt every disk, store keys in the keychain, use a dedicated test provider account, and back up *findings-registry.jsonl*.

6.3 SSO and identity

Sparfuchs QA does not issue its own identity. Authentication to the LLM providers is per-key, and authentication to the target repo is delegated to *git*. SSO integration is therefore a property of the provider account (for example, SSO-protected Anthropic console accounts) and of the git host (SSO-protected GitHub Enterprise), not the toolkit.

6.4 Multi-factor authentication

MFA is handled at the provider level. Recommended posture:

- Provider dashboards behind SSO+MFA.
- Long-lived API keys rotated every 90 days.
- Short-lived OAuth tokens preferred where possible.
- Keychain-stored keys protected by the local user's login password.

6.5 API key management lifecycle

Action	Frequency	Command
Rotate provider keys	Every 90 days	Generate new, *security delete-generic-password* + *security add-generic-password*.
Audit who has keys	Monthly	Cross-check provider console audit logs against operator list.
Revoke on offboarding	Within 24 hours	Revoke the provider-side key; do NOT rely on workstation wipe alone.
Audit approved providers list	Quarterly	Review *approvedProviders* in *models.yaml*.

6.6 Agent sandboxing and containment

This is the security argument for running Sparfuchs QA in your supply chain. Agents are not trusted code; they are instructions to an LLM. The toolkit surrounds them with six independent containment layers so that a compromised or hallucinating agent cannot exfiltrate, mutate, or escalate. You can safely forward this section to a security reviewer as a standalone artifact.

6.6.1 Tool allow-list per agent

Every agent markdown declares its allowed tools in frontmatter. The orchestrator enforces that list at runtime; an agent that tries to invoke a tool it has not declared fails with a loud error. The core set is *Read*, *Grep*, *Glob*, *Bash*, *Write*, *Edit*, and *WebFetch*. Plus MCP tools declared in *config/*.

6.6.2 Filesystem jail

Reads and writes are confined to three roots: the target repo, the sparfuchs-qa workspace (*qa-data/*, *qa-reports/*, *generated/*), and */tmp/* for credential files. Symlinks are resolved and their destinations re-checked against the jail. *Write* and *Edit* are blocked on the target repo entirely; agents cannot modify the code they are reviewing.

6.6.3 Bash command allow-list

The *Bash* tool, when available, is gated by *block-dangerous-commands.sh*. Destructive commands are denied before dispatch. Specifically:

Blocked command	Reason
`rm -rf` on any absolute path	Mass deletion
`git push`, `git push --force`	No write to remote
`git reset --hard`	Destroys working tree
`git checkout .`, `git restore .`	Discards operator work
`git clean -fdx`	Removes untracked files
`git branch -D`, `git tag -d`	Destructive ref operations
`sudo`, `doas`	Privilege escalation
`curl`, `wget` to unapproved hosts	Egress control
`chmod 777`, `chown root`	Permissions escalation

6.6.4 Data classification routing

The *dataClassification* setting in *models.yaml* gates which providers may receive repo data at agent execution time:

- **public:** all enabled providers are candidates.
- **internal:** only providers on the *approvedProviders* list.
- **restricted:** external API providers disabled entirely. Only local CLI providers (*claude-cli*, *gemini-cli*, *codex-cli*, *openclaw*) are eligible. Source code never leaves the workstation.

6.6.5 Hook-based pre-write scanning

Before any write reaches disk, the pre-write hook chain runs:

- *scan-secrets.sh* scrubs the content for API keys, tokens, private keys, and common connection strings.
- *protect-files.sh* refuses writes to *.env*, **.pem*, **.key*, lockfiles, and *.git/* anywhere on the host.
- *warn-large-files.sh* flags reads that exceed the Read tool's 2000-line window, so agents cannot silently work off truncated content.

6.6.6 Agent integrity validation

On load, every agent file's SHA-256 hash is compared to *config/agent-hashes.json*. Mismatches emit a warning today (planned hard block in v1.1, see gap 4.5). Administrators should commit the hashes file alongside every agent edit, and should treat an unexpected mismatch on an operator host as a supply-chain event.

6.6.7 Delegation rules we recommend

1. For any agent that should never shell out, set *disableBash*: *true*. This caps its tool surface at Read, Grep, Glob, and Write-to-workspace.
2. For any agent that makes network calls (only *ref-doc-verifier* today), require *dataClassification*: *internal* or *restricted*.
3. For cross-domain agents (*release-gate-synthesizer*, *workflow-extractor*), pin to Grok via *agentOverrides*. Honesty-critical agents do better on low-hallucination models.
4. Review the allow-listed Bash set (6.6.3) quarterly. If your environment runs unusual CI commands, extend the list explicitly rather than loosening it.

6.7 What agents should never be allowed to do

- Commit, tag, or push to any git remote. Enforced by the Bash allow-list.
- Open a pull request, merge, or rebase. No MCP tool for this is installed.
- Modify *config/models.yaml* at runtime. The orchestrator reads it once on startup and does not write to it.
- Read or write *.env*, private keys, or lockfiles. Enforced by *protect-files.sh*.
- Write to the target repository. Enforced by the filesystem jail.
- Make outbound network calls from within Bash. The allow-list denies *curl* and *wget* to unapproved hosts.

7. Monitoring, logging, and observability

Sparfuchs QA instruments itself heavily. This chapter tells the administrator which metrics to expose to an external dashboard, how to set alert thresholds, and how to interpret the quality auditor's verdicts.

7.1 What gets measured

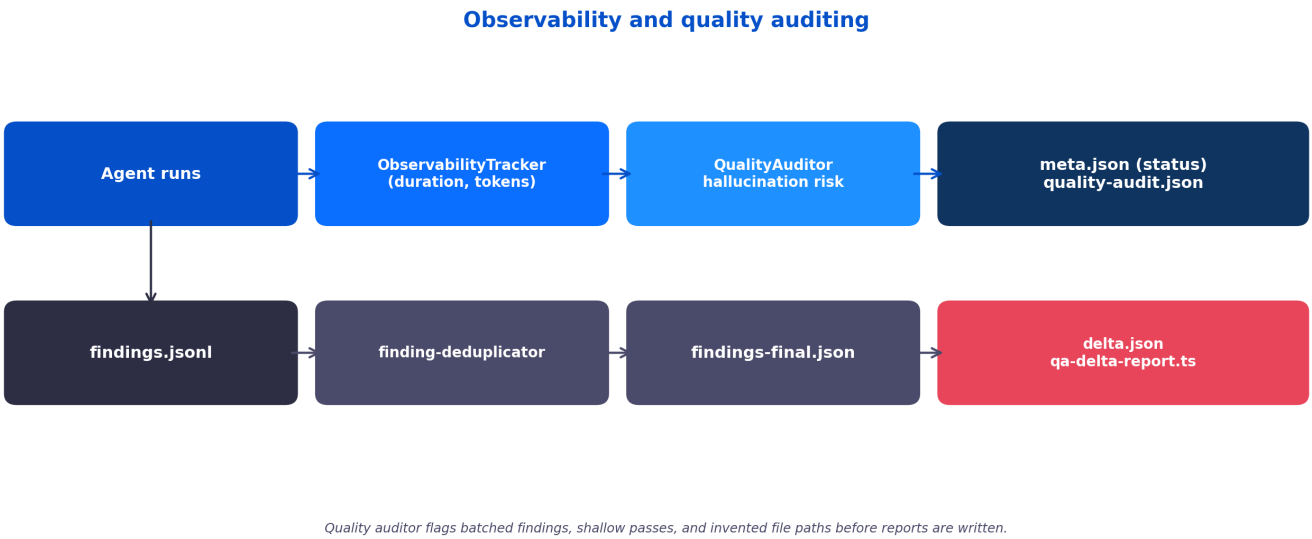

Figure 7.1 Observability and quality auditing pipeline.

Metric	Source	Typical use
Agent duration (ms)	observability.ts	Identify slow agents that exceed the step budget.
Tokens in / out	observability.ts	Track cost per run; enforce budgets.
Tool call count	observability.ts	Low counts trigger the give-up detector.
Findings raised	findings-final.json	Watch for unusually large or small counts.
Hallucination risk score	quality-audit.json	Flag agents that exceed your threshold.
Coverage per agent per chunk	coverage-report.json	Identify files that were skipped.
Status (succeeded / partial / errored)	meta.json	Primary alerting signal.

7.2 Dashboards and external tools

The orchestrator does not ship a dashboard. Emit *RENDERER=json* events to your existing log aggregator and build dashboards there. Our reference panels:

- **Runs over time**, stacked bar by status (succeeded / partial / errored).

- **Duration heatmap**, agents on Y axis, days on X axis.
- **Token cost**, summed across runs per provider.
- **Findings by severity**, weekly trend.
- **Quality flags**, count of batched / give-up / small-output incidents.

7.3 Alerting thresholds

Signal	Threshold	Suggested action
Run status = errored	Any occurrence	Page the administrator.
qualityFlags includes giveUp	2 or more in 24h	Investigate the offending agent + provider pair.
Token spend	> 120% of rolling 7-day baseline	Pause and audit; possibly cap with *defaultCap*.
Agent duration	> 2x rolling median	Raise *maxSteps* or switch provider.
Findings count	< 10% of baseline	Possibly a broken run. Re-run with *FULL=1*.

7.4 Agent-specific monitoring

Four agents deserve dedicated dashboards and explicit alert thresholds. We cannot measure a truly missed finding by definition (a missed finding is one we never saw), but we can monitor the proxies that correlate with one: low tool call counts, small outputs, and quality-auditor flags.

Agent	Why it matters	Alert on
security-reviewer	Highest blast radius.	qualityFlags contains *giveUp* or *smallOutput*; tool calls < 8; hallucination-risk > 60.
release-gate-synthesizer	Its verdict drives deploys.	BLOCKED rate rises above 15% of runs over 7 days; verdict flips within a single sprint.
qa-gap-analyzer	Trend tells you if the team is keeping up.	Coverage gap count rises 3 weeks in a row; new gap categories appear.
build-verifier	Failure cascades: later agents skip.	Early-abort rate above 10% of runs; specific error signature recurs.

Note. Missed findings cannot be measured directly; you catch them retroactively when an incident post-mortem references code Sparfuchs QA had reviewed. Log the agent and run id in your post-mortem template so you can feed those cases back into your agent tuning.

7.5 Anomaly detection

The *qa-evolve-v2* script is an administrator's friend. It reads 30 days of run history and flags agents whose thresholds should tighten (consistently zero findings) or whose thresholds should widen (noisy false positives). Run it monthly:

```
make qa-evolve-v2 PROJECT=my-app
```

8. Security, compliance, and governance

This chapter concentrates the security model in one place. It is the first thing to read if you are responsible for answering a compliance questionnaire or investigating an incident.

8.1 Security model summary

- **Static analysis, not runtime.** The toolkit reads files, runs grep, and calls an LLM. It does not execute the target application.
- **Read-only by default.** Write permissions apply only to the sparfuchs-qa workspace. The *protect-files.sh* hook blocks writes to *.env*, **.pem*, **.key*, lockfiles, and *.git/* anywhere.
- **OS-level secrets.** API keys in the OS keychain. Credential files in */tmp* with 0600 mode and auto-cleanup.
- **Integrity validation.** Agent SHA-256 hashes checked at load time.
- **Data classification.** *public* / *internal* / *restricted* routing enforced at agent execution time.

8.2 Encryption and data handling

Data	At rest	In transit
API keys	OS keychain (encrypted)	TLS 1.2+ to provider endpoints.
Test credentials	/tmp/...creds-{runId}.json, mode 0600	Passed to agents via env var *SPARFUCHS_CRED_FILE*.
Source files	Disk, standard OS ACLs	TLS 1.2+ to LLM providers (unless restricted).
Reports + findings	Disk, standard OS ACLs	Not transmitted.
Session logs	Disk, standard OS ACLs	Not transmitted.

8.3 Audit logging

Every run writes a *session-log.md* with the full per-agent transcript. Retention is a local policy decision. We recommend:

- Keep session logs for 90 days, then move to cold storage.
- Delete temp credential files immediately via the cleanup trap.
- Back up *findings-registry.jsonl* monthly, so audit trails survive workstation wipes.

Warning. Session logs are not automatically purged. If your target repo has PII or regulated data, either avoid reviewing it (use *dataClassification: restricted*) or purge session logs to a retention schedule appropriate to your regulator.

8.4 Compliance posture

Sparfuchs QA is a tool, not a certification body. It does not issue SOC 2, HIPAA, or PCI-DSS attestations. What it can do:

- Run the *compliance-reviewer* agent over a target repo, surfacing GDPR, CCPA, and PII-handling issues in source code.
- Enforce *dataClassification: restricted* for regulated repos.
- Produce a findings-registry that can be exported as evidence of regular review.
- Integrate with SBOM tooling via *make qa-sca*.

8.5 Known gaps and what to do about them today

The *docs/CAVEAT-EMPTOR.md* document is required reading. We list the highest-severity gaps here along with the concrete interim action the administrator should take before the fix ships. Honesty about gaps is a feature of this toolkit; so is giving you a playbook.

#	Gap	Severity	Status	What to do today
4.1	Plaintext creds in /tmp	CRITICAL	Missing, planned v1.1	Do not run reviews on untrusted workstations. Ensure /tmp is on an encrypted volume (FileVault, BitLocker, LUKS). Prefer API-key providers that do not need the wizard.
4.2	No PII masking in outputs	HIGH	Missing, planned v1.1	Do not review repos that contain hardcoded PII. Use *dataClassification: restricted* as an interlock on sensitive hosts.
4.3	No prod env detection	HIGH	Missing, planned v1.2	Operator policy: never store production DB or admin creds in the sparfuchs-qa keychain. Use a dedicated test account per target repo.
2.1	No runtime analysis	HIGH	By design	Pair Sparfuchs QA with integration and E2E tests. It catches static issues, not runtime behavior.
2.2	No business logic verification	HIGH	By design	Keep hand-written tests with known expected values for any pricing, billing, or regulated logic.
4.5	Advisory integrity check	MEDIUM	Weak, planned v1.1	After every intentional agent edit, run *make qa-hashes-update*. Treat unexpected integrity warnings as a supply-chain event.
4.6	LLM false confidence	MEDIUM	Partial mitigation	Cross-check Critical findings in the session log before escalating. Set *agentOverrides* to route honesty-critical agents to Grok.

Tip. A director briefing this tool to a security team should lead with this table. It shows the tool is honest about its limits AND gives a concrete mitigation for every one.

8.6 Risk assessment and escalation

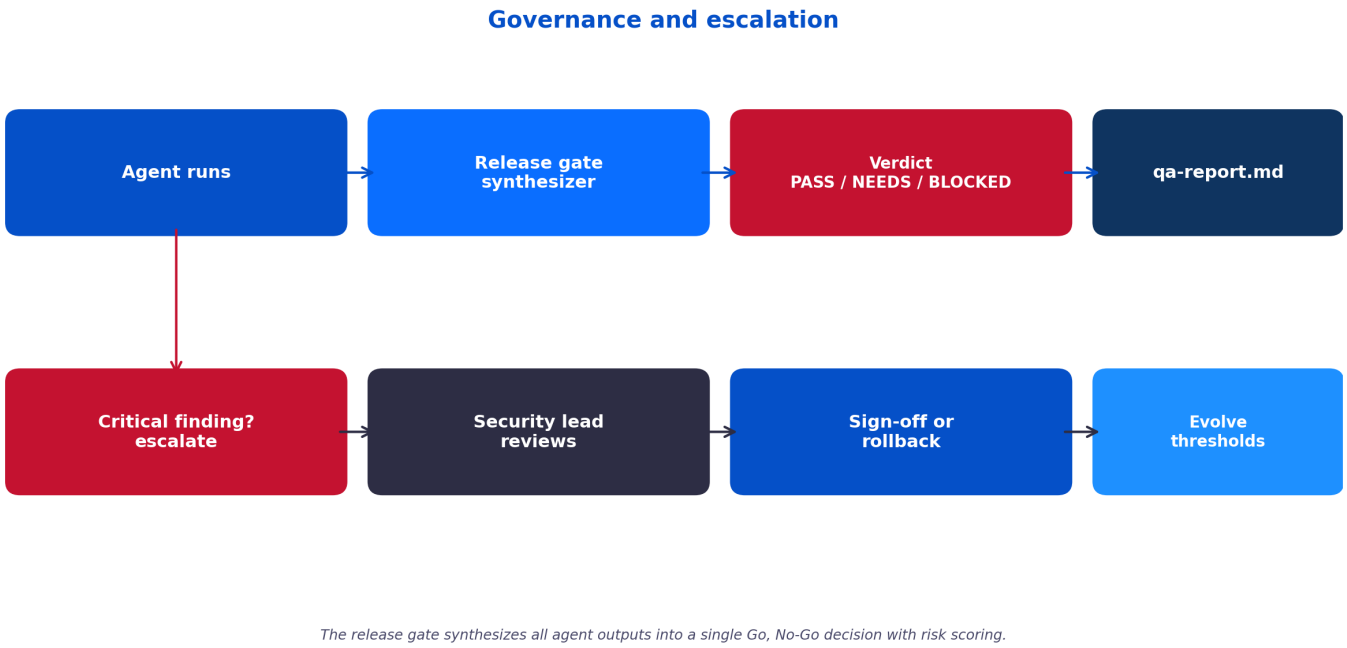

Figure 8.1 Governance and escalation flow.

When the release gate issues a **BLOCKED** verdict, the administrator should act within one business day. **NEEDS CHANGES** verdicts should be resolved within the sprint. **PASS** verdicts can be archived.

8.7 Oversight policies for agentic behavior

- Every agent action must be traceable through the session log.
- Agents may not autonomously commit, tag, push, or open a PR. This is enforced by the hooks and by the lack of Write tooling on the target repo.
- Agents may not modify the *config/models.yaml* at runtime.
- Generated tests are drafts and must be human-reviewed before being committed.

8.8 Anti-hallucination layers

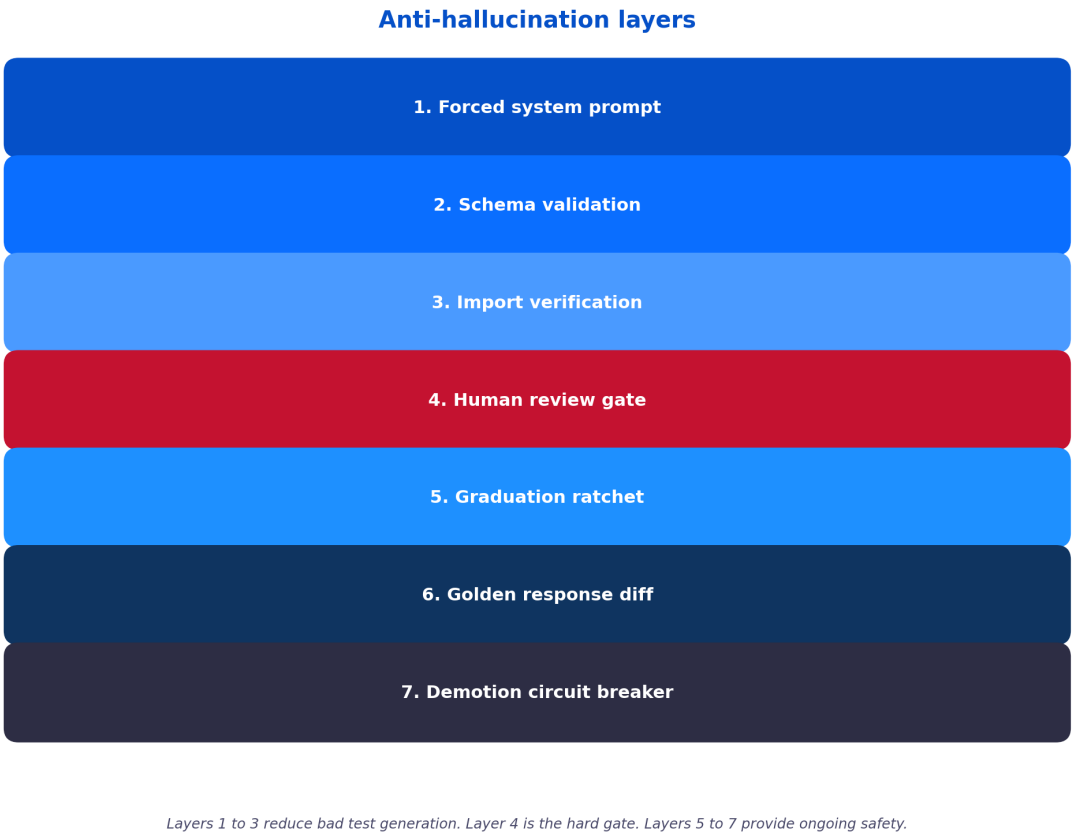

Figure 8.2 Anti-hallucination layers.

9. Maintenance and operations

Keep Sparfuchs QA healthy with a small set of recurring tasks. This chapter lists them at daily, weekly, monthly, and quarterly cadences.

9.1 Maintenance cadence

Cadence	Task	Command / note
Daily	Run canaries on the sparfuchs-qa repo itself	`make qa-quick`
Daily	Monitor alerts from RENDERER=json event stream	See chapter 7 thresholds.
Weekly	Review *qa-delta PROJECT=…* for regressions	`make qa-delta PROJECT=my-app`
Weekly	Check *make qa-keys-check* on each operator host	Keys drift over time.
Monthly	Run *qa-evolve-v2* and review threshold suggestions	`make qa-evolve-v2 PROJECT=...`
Monthly	Archive runs older than 30 days	`make qa-cleanup PROJECT=...`
Monthly	Rotate any API keys older than 90 days	Use per-platform commands from chapter 5.
Quarterly	Re-read CAVEAT-EMPTOR.md for new gaps	
Quarterly	Regenerate agent hashes after intentional edits	`make qa-hashes-update`
Quarterly	Review approved providers list	Edit *models.yaml*.

9.2 Backup and restore

What to back up:

- *config/models.yaml* and *config/agent-hashes.json*, because they are the current configuration.
- *qa-data/<project>/findings-registry.jsonl* for every project, because they encode long-lived finding lifecycle decisions (wont-fix rationales, closed evidence).
- *.claude/agents/* and any locally customized agents.

What does not need backup:

- Per-run bundles. They are reproducible from source and an LLM re-run.
- Session logs older than your audit retention window.
- *node_modules/*. It is regenerated by *npm ci*.

9.3 Updates and patching

The update flow is plain git plus npm:

1. On a throwaway branch, *git pull* the latest changes.
2. Run *npm ci* to install matching dependencies.
3. Run *make qa-quick*. All 15 canaries must pass.
4. Run *make qa-hashes-update* if any agent markdowns changed.
5. Run *make qa-verify* to re-check supply-chain attestations.
6. Commit, tag the release, roll out to operator hosts.

9.4 Zero-downtime rollouts

Because there is no service, there is no downtime. Roll out updates to operator hosts on a staggered schedule. Provide a ten-minute "did anything break?" window between waves.

9.5 Scaling considerations

When a project is large enough that *FULL=1* takes longer than the operator is willing to wait, the knobs in order of effectiveness are:

1. **Module scoping.** Review one app at a time with *MODULE=apps/x*.
2. **Coverage strategy.** Downgrade from *exhaustive* to *thorough* or *balanced*.
3. **Agent preset.** Use *standard* instead of *full*.
4. **Provider tier.** Pin *light* for agents that are good enough at that tier.
5. **Chunk size.** Increase chunk size in the testability scanner to reduce overhead per chunk.

9.6 Disaster recovery

The workstation loss case is handled by your normal laptop backup policy. What Sparfuchs QA adds to the discussion:

- Keychain contents are tied to the user's login credentials. They do not survive a wipe. Back up provider keys to a secrets manager outside the workstation.
- *findings-registry.jsonl* is the only long-lived artifact that matters. Mirror it to a network share weekly.
- Generated tests in *generated/* should be copied into the target project's test suite as soon as they are approved, so they survive a sparfuchs-qa wipe.

9.7 Updating agent logic and prompts

Agent markdowns live in *.claude/agents/*. To tune an agent:

1. Edit the markdown. Save.
2. Run *make qa-hashes-update*. This regenerates *config/agent-hashes.json*.
3. Run a targeted review against a stable corpus repo to diff findings before and after the change.
4. Commit the agent + the hashes file together.

Warning. Never commit an edited agent without the matching hashes file update. The integrity check will fire on every future run and add noise.

10. API, integration, and extensibility

Sparfuchs QA exposes no HTTP API. Integration happens through file formats, env vars, CLI flags, and adapter plug-ins. This chapter is a reference for the administrator or developer who wants to extend the toolkit.

10.1 Artifact formats

File	Format	Contract
meta.json	JSON	*status*, *agents*, *coverage*, *qualityAudit*, *isGitRepo*.
findings.jsonl	JSONL, one per line	Streamed per-agent findings (see *QaFinding* type).
findings-final.json	JSON array	Canonical deduplicated findings (schema in lib/types.ts).
delta.json	JSON	*newFindings*, *recurringFindings*, *remediatedFindings*, *closureRate*, *regressionRate*.
coverage-report.json	JSON	Per-agent, per-chunk coverage tracking.
quality-audit.json	JSON	Quality auditor flags per agent.
agent-data/<agent>.output.json	JSON	Per-agent handoff envelope.

10.2 The JSON handoff contract

Inter-agent communication is JSON-only. An agent emits findings to *findings/<agent>.json* (array of *QaFinding*) and handoff data to *agent-data/<agent>.output.json* (object). Malformed JSON aborts the run with *AgentIngestionError*, naming the offending agent and run id. There is no silent-skip.

Legacy HTML-comment tags (*<!-- finding: {...} -->*, *<!-- agent-data: {...} -->*) remain as a fallback, but they also fail loudly on parse errors.

10.3 Writing a new agent

1. Create *.claude/agents/my-agent.md*.
2. Include frontmatter with *description*, *model* tier (opus / sonnet / haiku), and *tools* (Read, Grep, Glob, Bash, etc.).
3. Write the prompt body. Be explicit about input files, output format, and limitations.
4. Define the agent's output shape: a JSON array of findings and / or an *agent-data/* handoff object.
5. Run *make qa-hashes-update*.
6. Add the agent to *scripts/qa-review-remote.sh*'s standard deployment list if it should run by default.

7. Test with *make qa-review REPO=… AGENTS="my-agent"*.

10.4 Writing a new adapter

Adapters live in *lib/orchestrator/adapters/*. To add a new provider:

1. Create *lib/orchestrator/adapters/my-provider-adapter.ts*.
2. Implement the *Adapter* interface: *detect()*, *run(agent, input)*, and the *capabilities* getter.
3. Register the adapter in *lib/orchestrator/adapters/index.ts*.
4. Add a provider block to *config/models.yaml*.
5. Run *npx tsc --noEmit* to catch type errors.
6. Run *make qa-review* with *ENGINE=orchestrated PROVIDER=my-provider*.

10.5 Event streams for dashboards

Set *RENDERER=json* to emit one event per line on stderr. Event types:

Event	When	Notable fields
runStart	Run begins	*runId, project, mode*
agentStart	Agent begins	*runId, agent, provider, tier*
agentProgress	Tool call emitted	*runId, agent, tool, stepIndex*
agentEnd	Agent finishes	*runId, agent, status, tokensIn, tokensOut, findings, qualityFlags*
runEnd	Run finishes	*runId, status, verdict*

10.6 Webhooks and integrations

There is no built-in webhook. Wire it in yourself: pipe JSON events to a process that posts to your webhook target. Common integrations:

- Slack, posting *runEnd* events to a channel when *status != succeeded*.
- Jira, creating issues from Critical findings.
- GitHub Checks, posting the release gate verdict as a PR check via the GitHub API.

10.7 Plugins and custom tools

The orchestrator supports MCP-style custom tools declared in *config/*. To add one, declare it in an agent's frontmatter and implement the handler in *lib/orchestrator/tool-implementations.ts*. Custom tools run in the same permission envelope as the core tools.

11. Advanced troubleshooting and diagnostics

Chapter 6 of the User Guide covers common operator-facing problems. This chapter covers diagnostics that require administrator access.

11.1 Log analysis playbook

1. Find the run: *ls qa-data/<project>/runs/ | tail -1*.
2. Inspect meta: *cat qa-data/.../runs/<runId>/meta.json | jq .status*.
3. Read the failing agent's block in *qa-reports/.../session-log.md*.
4. Cross-check *quality-audit.json* for quality flags on that agent.
5. Look at *coverage-report.json* to see which files were actually read.
6. Reproduce with a targeted run: *make qa-review REPO=... AGENTS="<failing-agent>"*.

11.2 Common failure scenarios

Scenario	First hypothesis	Next step
Agent crashes halfway	Context overflow	Raise *maxSteps* and / or switch to a heavier tier.
All agents skipped	Testability preflight judged repo uncheckable	Read the preflight summary in *meta.json*.
Provider returns 429 mid-run	Rate limit, most common transient failure	The fallback chain catches most of these automatically. If it does not: reduce *coverageStrategy* one step (exhaustive -> thorough -> balanced), switch the run to the *standard* preset, or pause and retry after the provider's documented cooldown.
Provider returns 401	Key missing or expired	Run *make qa-keys-check*.
Integrity mismatch warnings	Unacknowledged agent edit	*make qa-hashes-update*.
Cleanup trap did not fire (SIGKILL)	Agents left in target repo	Manually remove; see User Guide section 6.5.
Disk filling up	Too many runs retained	*make qa-cleanup PROJECT=...*.

11.3 Performance bottlenecks

- **Slow LLM calls**: switch to a lighter tier for non-honesty-critical agents.
- **Large-file reads**: the Read tool caps at 2000 lines. The testability scanner already warns; honor its recommendations.
- **Grep result truncation**: cap at 250 matches. Narrow patterns or split work.
- **Too many agents**: use the *standard* preset and add agents back one at a time until the duration becomes unacceptable.

11.4 Debug modes

```
# Debug a single agent
npx tsx lib/orchestrator/debug-single-agent.ts --agent security-reviewer \
--repo /path/to/project

# Full verbose logging
DEBUG=sparfuchs:* make qa-review REPO=/path/to/project

# Pretty-print the event stream
RENDERER=json make qa-review REPO=/path 2> >(tee /tmp/events.jsonl | jq .)
```

11.5 Support bundle creation

When filing a bug, attach the support bundle:

```
# Redacted support bundle
tar -czf support-bundle-$(date +%Y%m%d).tgz \
--exclude='*credential*' \
--exclude='.env*' \
qa-data/<project>/runs/<runId>/ \
qa-reports/<date>_<project>_*.md \
config/models.yaml \
config/agent-hashes.json
```

Warning. Always audit the tarball before sending. Run *tar -tzf support-bundle.tgz* and scan for *.env*, **.pem*, *credentials*, *secrets*, or references to production hostnames. Delete anything questionable before uploading.

12. Appendices

Appendix A. Sample models.yaml

```
defaultProvider: xai
coverageStrategy: balanced
dataClassification: internal
redactSecrets: true
approvedProviders: [anthropic, google]

providers:
xai: { type: api, enabled: true, apiKeyEnvVar: XAI_API_KEY }
google: { type: api, enabled: true, apiKeyEnvVar: GOOGLE_GENERATIVE_AI_API_KEY }
anthropic: { type: api, enabled: true, apiKeyEnvVar: ANTHROPIC_API_KEY }
openai: { type: api, enabled: false, apiKeyEnvVar: OPENAI_API_KEY }
claude-cli: { type: cli, enabled: auto, binary: claude }
gemini-cli: { type: cli, enabled: auto, binary: gemini }

fallbackChain: [anthropic, google, claude-cli, gemini-cli]

tiers:
heavy: { anthropic: claude-opus-4-6, google: gemini-3.1-pro-preview }
mid: { anthropic: claude-sonnet-4-6, google: gemini-3-flash-preview }
light: { anthropic: claude-haiku-4-5, google: gemini-3.1-flash-lite-preview }

agentOverrides:
security-reviewer: { provider: anthropic }
ui-intent-verifier: { provider: anthropic, disableBash: true }
observability-auditor: { provider: google, maxSteps: 75 }
ref-doc-verifier: { provider: google, maxSteps: 100 }
```

Appendix B. Reference tables

B.1 Ports and network endpoints

The toolkit binds no ports. All traffic is outbound HTTPS on TCP/443 to LLM provider endpoints; see chapter 2.4 for the destination list.

B.2 Exit codes

Code	Meaning
0	Success.
1	Canary or build failure.
2	Configuration error (missing REPO, invalid flags).
3	Authentication failure (no provider key, no CLI auth).
4	Agent ingestion error (malformed JSON).
5	Cleanup failure (deployed agents could not be removed).
130	Interrupted by Ctrl-C (cleanup trap ran).

B.3 Severity rubric

Severity	Criteria	Typical action
Critical	Security exploit, data loss, or deploy-breaking regression	Block release; fix immediately.
High	Significant risk, not exploitable today	Fix in the current sprint.
Medium	Moderate code smell or risk	Schedule in the backlog.
Low	Minor issue, easy to remediate	Opportunistic fixes.
Info	Notable but not actionable	Archive.

Appendix C. Audit evidence checklist

- Last 12 months of *qa-runs* metadata exported to JSON.
- *findings-registry.jsonl* for each active project.
- Current *config/models.yaml*.
- Current *config/agent-hashes.json*.
- Last 90 days of session logs (redacted).
- Keychain audit: list of accounts under service *sparfuchs-qa*.
- Policy document: agent override policy, data classification policy, retention policy, rotation policy.

Appendix D. License information

The Sparfuchs QA source code is distributed under the terms described in the *LICENSE* file at the repo root. Third-party dependency licenses are reported by *make qa-sca*, which generates a CycloneDX-lite SBOM and runs *npm audit* for known CVEs.

Appendix E. Vendor contacts

Vendor	Product	Contact
Anthropic	Claude API, Claude Code CLI	support@anthropic.com
Google	Gemini API, Gemini CLI	Google Cloud support
xAI	Grok API	support@x.ai
OpenAI	GPT API	help.openai.com
Sparfuchs Corporation	This toolkit	qa-admin@sparfuchs-pro.com

Appendix F. Changelog

Version 1.0, April 22, 2026

- First published edition.
- Architecture, installation, configuration, credentials, RBAC, monitoring, security, maintenance, API, troubleshooting, appendices.
- Based on sparfuchs-qa repo state as of 2026-04-13.

Planned for v1.1

- Air-gapped deployment playbook.
- Hardening guide for the *restricted* data classification.
- Sample Grafana / Datadog dashboards.

Note. This roadmap is advisory and not a commitment. Features and dates may change without notice. For the current feature set, refer to the most recent release notes.

Appendix G. Hardening roadmap

Lifted from *docs/CAVEAT-EMPTOR.md* so the administrator can plan investment. This roadmap evolves, always consult the source doc.

Phase	Items	Horizon
Phase 1	Encrypt credential files at rest; PII redaction; hard-block agent integrity; session log retention policy	1-2 weeks
Phase 2	Production environment detection; per-agent filesystem jail scoping; test account tagging and cleanup; large-file chunked reading heuristic	2-4 weeks
Phase 3	Finding validation rate tracking; data-flow tracer agent; expanded canary patterns; coverage tracking with retry	1-2 months

Appendix H. Documentation maintenance plan

The Admin Guide is owned by the Platform team. Update cadence is quarterly for normal changes and immediate for any change that alters the security posture, credential flow, or configuration schema. Each release includes a documentation diff; the release-gate-synthesizer flags *NEEDS CHANGES* if the guide has drifted. Owners: Platform team lead (primary), Security lead (review).